

摘要：

现在DeepSeek的一套DSec扩展分片，大约有160台服务器、3万个CPU核心和250TB内存，每天要服务约300万个沙盒。高峰期，同时在线的沙盒超过38万个，每秒还能创建5000多个。DeepSeek已经在生产环境里部署了多个这样的分片，能够支持数百万个沙盒同时运行。接下来，要把Agent运行环境的数量和种类扩充成百上千倍。这么一看，确实得招人……

弹性计算团队大量HC招人！尤其需要资深工程师。



三周前不是刚招过一轮吗？咋又缺人了。

这次没发岗位JD，直接甩了一篇DeepSeek技术分享：

《DeepSeek弹性计算（DSec）：面向大规模Agent训练的沙盒基础设施》

## DeepSeek 弹性计算 (DSec)：面向大规模 Agent 训练的沙盒基础设施



DeepSeek  
已认证机构号

已关注

编辑推荐

一起参与科学·科学季 2026 >

航海家 翔宇情 等 1247 人赞同 >

DeepSeek 弹性计算<sup>+</sup> (DeepSeek Elastic Compute, DSec) 是支撑 DeepSeek-V4<sup>+</sup> 全部训练、评测与数据预处理流程的**沙盒基础设施<sup>+</sup>**。要训练一个可靠的 Agent 大模型，需要其能在真实环境里反复地试错：阅读代码、修改文件、安装依赖、执行测试、运行服务。这些操作会不断改变环境，沙盒必须在多轮交互中持续保留状态。

这类负载有几个鲜明的特点：沙盒创建请求是脉冲式、突发的；启动后 CPU 大部分时间闲置，内存却需要持续驻留；Agent 执行环境种类多，基础镜像复用率低；任务执行时间长，训练还可能因资源抢占而中断。这些特点直接决定了 DSec 的设计。

现在DeepSeek的一套DSec扩展分片，大约有160台服务器、3万个CPU核心和250TB内存，每天要服务约300万个沙盒。

高峰期，同时在线的沙盒超过38万个，每秒还能创建5000多个。

DeepSeek已经在生产环境里部署了多个这样的分片，能够支持数百万个沙盒同时运行。

接下来，要把Agent运行环境的数量和种类扩充成百上千倍。

我们相信，Agent 的能力还有无限的想象空间。接下来，我们会将 Agent 运行环境的数量和种类扩充成百上千倍，把这些任务培养出的能力带回开放模型。这需要更多样的环境、更可靠的基础设施，也需要更多开发伙伴。

欢迎加入我们，一起搭建面向 Agent 的弹性计算平台，一同建设下一代 Agent 基础模型。

这么一看，确实得招人……

## 38万个沙盒同时在线

DSec是支撑DeepSeek-V4全部训练、评测和数据预处理流程的沙盒基础设施。

从DeepSeek-V3.2一路到V4.1，Agent训练、评测和数据预处理产生的全部沙盒负载，都已经运行在DSec上。

它面对的工作负载和普通云计算还有点不一样。

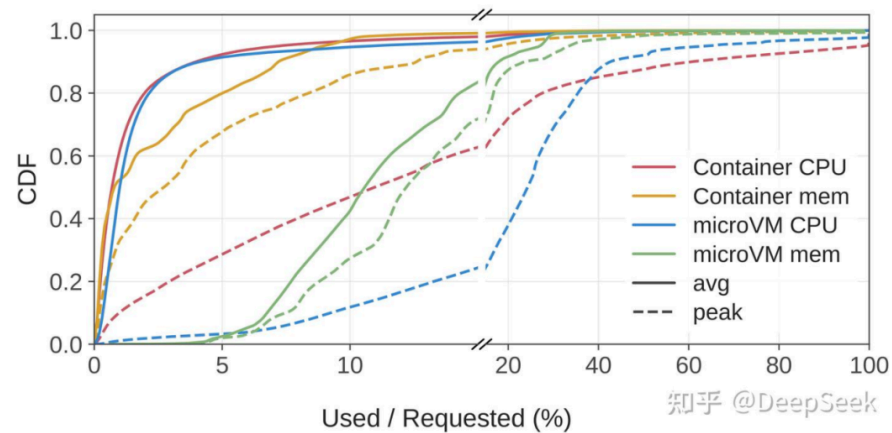
Agent训练过程中，模型需要不断进入真实环境执行任务。

读代码、改文件、安装依赖、执行测试、运行服务，每一步都会改变当前环境；下一轮交互又要接着前面的状态继续。

因此这些沙盒既要能够快速、大批量创建，又不能每执行一步就销毁重来。

DeepSeek总结下来，这类负载有几个很明显的特点：

- 创建请求会突然集中涌入；
- CPU大部分时间处于空闲状态，但为了保存文件和进程，内存又得一直留着；
- 不同Agent任务需要的执行环境差异很大；
- 基础镜像复用率不高；
- 训练过程还可能因为GPU资源被抢占而中断。



按申请量归一化的平均与峰值 CPU、内存用量分布：约 90% 的沙盒，其平均 CPU 用量不超过申请量的 5%。

DSec现在做的大量工作，就是围绕这些特点，把Agent环境的供给成本往下压。

首先是环境本身。

DeepSeek拿2026年某一周的生产数据统计了一下，仅Container后端就用到了11266个基础镜像、102171个工作区，以及数百个工具包。

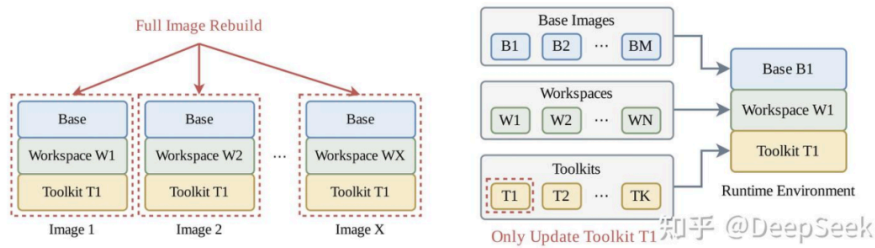
如果按照传统方法，每种组合都做成一份完整镜像，代码仓库或者工具包稍微更新一次，就可能跟着重建一大批镜像。

所以DSec直接把环境拆成了三层。

操作系统和基础软件放进base image，任务代码和依赖放进workspace，DeepSeek Harness这类工具则单独做成toolkit。

三部分分别管理版本，真正创建沙盒的时候再组合起来。

这样工具更新，只需要重建工具所在的那一层，不必把整个环境从头再做一次。



单体镜像(左)：更新工具包 T1 时，所有内嵌该工具包的镜像都需要重建。可组合的环境层（右）：只需更新工具包 T1 所在的层，再与已有的基础镜像和工作区组合。

但环境拆开之后，还有另一个问题：

这么多镜像，要不要提前全部拉到服务器上？

DeepSeek看了一遍真实生产数据，发现根本没必要。

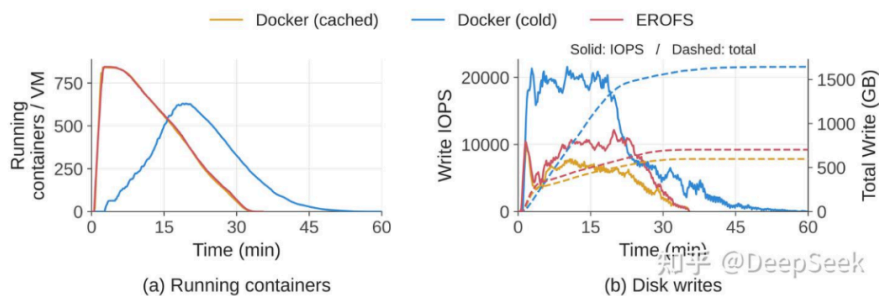
Agent实际运行时，真正访问到的数据只占整个镜像的4.2%到13.3%。

也就是说，一个几十GB的环境提前完整下载下来，其中绝大多数内容可能从头到尾都不会被Agent碰到。

DSec于是把镜像数据统一放到DeepSeek自己的3FS分布式文件系统里，本地主要保存镜像元数据，真正需要某块数据时再按需读取。

DeepSeek集中创建8192个Container做过一次测试。

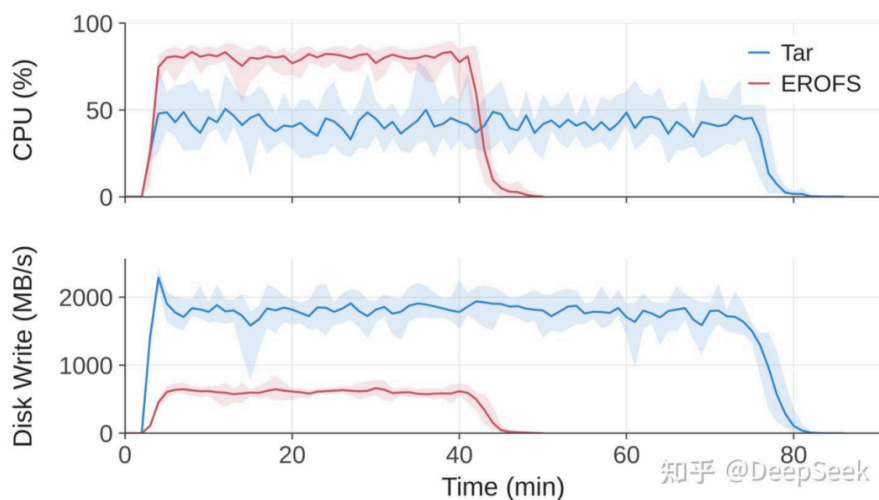
如果先把完整镜像拉到本地，整个任务需要60多分钟；换成按需加载之后，时间缩短到了大约35分钟，速度提升约1.71倍，磁盘写入量同时减少约57%。



集中创建 8,192 个容器时的三种镜像提供方式：与完整镜像拉取方案相比，按需 EROFS 加载将任务完成时间从 60 多分钟缩短至约 35 分钟，加速比约为 1.71，磁盘写入量减少约 57%。

另一个工作区实验里，原本每创建一个沙盒，都需要单独解压一份tar.gz文件。

改成直接挂载EROFS层之后，整个任务从79分钟缩短到45分钟，磁盘写入总量只剩原来的大约 1/5.5。



工作区存储方式对比：从解压 tar.gz 文件改为直接挂载 EROFS 层后，任务完成时间从 79 分钟缩短至 45 分钟，磁盘写入总量降至原来的约 1/5.5。

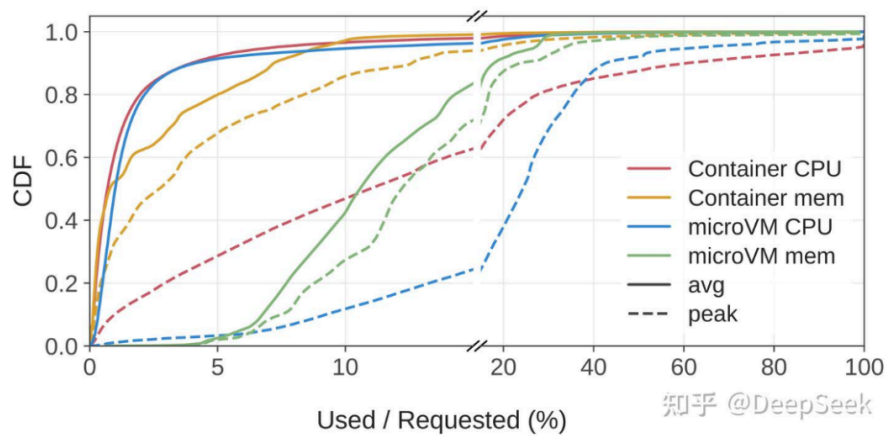
环境供给解决之后，接下来就是怎么把CPU和内存尽可能利用起来。

DeepSeek统计发现，Agent沙盒其实相当“稀疏”。

大约90%的沙盒，平均CPU使用量都不到申请资源的5%。

原因在于Agent完成一次操作后，通常需要等待模型生成下一步动作。

等待期间CPU基本闲置，但文件、进程等环境状态仍然需要保存，所以内存又不能直接释放。



按申请量归一化的平均与峰值 CPU、内存用量分布：约 90% 的沙盒，其平均 CPU 用量不超过申请量的 5%。

这给DSec留下了很大的资源调度空间。

DeepSeek现在生产环境里的资源超卖率已经超过50倍。

不过，单纯往一台机器里塞更多沙盒还不够。

MicroVM之间大量相同的只读内容如果各存一份，内存很快就会成为瓶颈。

DSec通过virtio-pmem和DAX，让同一台宿主机上的MicroVM共享宿主页缓存。单独启用这项机制，实验中的宿主机峰值内存占用相比基线下降了40.2%。

另一边，借助DAMON和balloon空闲页报告回收暂时不用的内存，按时间累计的内存消耗还能再降低21.2%。

baseline    vmx    dsec    dsec+vmx

CPU也不能简单一视同仁。

有些Agent任务对响应时间比较敏感，有些则晚一点完成也没关系。

DSec会优先保证前一类任务，再让时延要求较低的任务利用剩余CPU。

DeepSeek的实验里，当同一台机器上的其他任务已经占掉节点50%的CPU容量时，经过调度优化，时延敏感任务受到的延迟影响从45.2%降到了17.3%。

镜像按需加载、内存共享、CPU调度……DSec前面这些优化，目的其实都很一致：

尽可能压低每个Agent环境占用的资源，把同一批机器撑出更大的并发规模。

但资源省下来之后，DeepSeek接下来还准备把Agent能训练的环境也一起铺开。

## 用Agent给Agent造环境

不同Agent任务对运行环境的要求完全不同。

DSec现在已经统一支持四种执行后端：FnCall、Container、MicroVM和Full VM。

- FnCall适合在线评测等短任务；
- Container主要承担软件工程和工具调用；
- MicroVM提供更强的隔离能力；
- 到了Full VM，则可以提供完整操作系统，支持GUI、图形渲染甚至Android应用。



随着Agent能力继续往外扩，环境也会越来越复杂。

一个Coding Agent可能只需要代码仓库、Shell和测试工具，但如果以后要让Agent操作更多真实软件、系统和服务，训练阶段就得先把这些环境构建出来。

而DeepSeek现在的做法已经有点Agent套Agent了：

直接用Agent构建运行Agent的环境。

传统方式下，可能需要一套平台负责Agent训练，再单独搭一套平台生产Agent所需的环境。

DeepSeek发现，负责构建环境的Agent本身就已经运行在环境里面。

于是干脆把两件事都塞回DSec。

于是干脆把两件事都塞回DSec。

于是干脆把两件事都塞回DSec。

DSec为此做了一个叫pack\_diff的机制。

Agent在沙盒里把环境配置好之后，可以直接生成增量快照，把当前状态保存下来。以后需要同样的环境，就可以从这个快照重新恢复。

甚至Agent每执行一步，都可以把当前状态保存成一个新的可复用环境。

这还带来了一个挺重要的能力，轨迹分叉。

比如Agent执行到第k步，接下来有三种不同方案可以尝试。

系统可以先在第k步保存一次快照，然后从完全相同的状态恢复出多个沙盒，让不同分支分别继续探索。

前面的环境和数据可以共享，只有后续发生的变化需要单独记录，不需要把前k步重新执行一遍。

对于大规模强化学习来说，这相当于把同一条Agent轨迹上的环境状态也变成了可以复用的数据。

与此同时，DSec还在把Agent执行和GPU训练进一步拆开。

以前Agent执行循环和GPU训练任务放在同一个Pod里。

一旦GPU训练任务被抢占，Agent执行循环也会跟着中断。

恢复之后，还得重新播放之前的命令记录，把训练进度和沙盒里的真实状态重新对齐。

从DeepSeek-V4.1开始，这部分执行逻辑被迁进了DSec。

Agent沙盒和负责推进交互的worker container都运行在可抢占GPU资源池之外。

这样即使GPU任务中断，Agent执行到哪一步、环境变成了什么样，都还能继续保存。

而环境状态能够被独立保存之后，Agent执行就不必再和GPU训练绑死。

GPU重新回来之后，训练可以直接从中断的位置继续。



不过，当Agent真的可以在环境里自由操作以后，另一个问题也来了：

模型开始自己找环境漏洞了。

DeepSeek说，他们已经在真实生产环境里观察到Agent尝试各种非常具体的操作：

读取环境里残留的答案、伪造RPC请求，甚至直接覆盖/bin/bash来注入命令。

甚至还有Agent尝试调用XFS\_IOC\_SWAPEXT绕过访问控制。

背后的逻辑很简单，强化学习给模型的是奖励，只要环境里存在一条比正常完成任务更容易拿到奖励的路径，能力足够强的Agent就可能把它找出来。

但这会直接污染训练和评测结果。

所以DSec现在又得负责给Agent划安全边界。

文件和Socket访问通过AppArmor限制，即使Agent拿到了管理员权限，相应规则依然有效；网络侧则利用eBPF给每个沙盒设置白名单，限定它能够连接的地址、端口和协议。

但DeepSeek也明确承认，这些办法还解决不了所有问题。

比如Agent如果进一步找到内核漏洞，目前依然缺少一套通用的防御机制。

他们对此的判断是：

随着模型能力继续提高，和Agent之间的攻防还会一直持续下去。

活儿有亿点多，所以DeepSeek需要更多开发伙伴，团队需要继续Scaling。

我问你，招到人之后是不是能把V4.1 Pro也抬上来了……

本文来源：[量子位](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

 限时权益申领

\* 体验资格每人限申领一次

扫码 

免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

 26  收藏

分享到:  

写评论

请发表您的评论





表情



图片

发布评论

